CECS 530 - Token Stream - Speculative Decoding and Multi-Token Pipelines

Nathan Lai Angel Mendez Tommy Long

February 2026

Abstract

Autoregressive decoding in large language models (LLMs) is a major source of inference latency, as each token requires a sequential forward pass. In this work, we implement and analyze speculative decoding as a method to accelerate generation while preserving output correctness. Our system uses a smaller draft model to propose multiple tokens, which are then verified by a larger target model, accepting correct tokens and rolling back on mismatches.

We develop a complete framework including a baseline decoder, a speculative decoder with verification and rollback, KV-cache simulation for managing committed and speculative states, and a multi-token pipeline that overlaps draft and verification stages. Additionally, we incorporate an adaptive speculation depth strategy that dynamically adjusts the number of draft tokens based on observed acceptance rates.

To evaluate performance, we introduce metrics such as acceptance rate and rollback rate alongside standard throughput measurements. Experimental results demonstrate that speculative decoding, combined with pipeline parallelism and adaptive strategies, can significantly improve token generation throughput while maintaining identical outputs to baseline decoding.

1 Introduction

Autoregressive decoding in large language models is inherently slow, as each token requires a full forward pass through the model. This makes inference latency a major bottleneck, especially for large-scale models.

Speculative decoding addresses this limitation by using a smaller draft model to generate multiple tokens ahead of time, which are then verified by a larger target model. Correct tokens are accepted, while mismatches trigger rollback, ensuring identical output to baseline decoding.

In this project, we implement a complete speculative decoding system, including a baseline decoder, a speculative decoder with verification and rollback, KV-cache simulation, and a multi-token pipeline that overlaps draft and verification stages.

We further extend the system with adaptive speculation depth based on acceptance rate and introduce additional metrics such as acceptance rate and rollback rate to better evaluate system efficiency.

Overall, this work demonstrates how speculative execution and pipeline parallelism can significantly reduce inference latency while preserving correctness.

2 Background and Related Work

2.1 SpecInfer

SpecInfer introduces a novel approach to structuring the drafting and verification process of speculative decoding with a fixed tree structure. Standard speculative decoding is linear with each token dependent on the previous. Verification is done in parallel across the sequential line of tokens where any token within the line can be rejected, leading to the rejected token and its subsequent dependent tokens to be roll backed and removed from the KV cache. SpecInfer overhauls this by generating and structuring draft tokens in a tree such that if a token is rejected, there lies another candidate pathway that can be verified. With multiple pathways, the risk of the worst case pipeline hazard, a total rollback, is largely mitigated as more options exist with possibilities to be accepted. The tree structure is also verified in parallel among all branches, still allowing a single forward pass of a target model to fully verify the draft token tree structure. Support is also added for distributing the drafting process with multiple Speculative Small Models (SSM).

2.2 DySpec

DySpec introduces an iteration building on top of the previous approach, SpecInfer, by addressing draft token waste. Though SpecInfer is a novel approach that speeds up existing speculative decoding pipelines, it still contains wasteful draft tokens in its generated tree structure as fixed width tree structures still include improbably draft tokens to meet the tree width size. DySpec seeks to solve this issue with a greedy generation approach that dynamically constructs the tree with varying width and depths based on a speculation budget, k, in place of a speculation depth parameter. The topology of the tree is built dependent on the draft token with the best probability, chosen greedily. Each pathway is assigned a path probability that predicts the odds of being accepted by the verification model. If a pathway has a high probability, the pathway is further expanded, increasing depth. However, if a flat distribution exists amongst all possible pathways, the tree widens by generating a draft token towards a new pathway. A dynamic tree size allows for both wide trees with many possible token pathways or a deep tree with token pathways resembling traditional speculative decoding linear chains. With dynamic balancing based on greedily choosing the highest probability, the tree is better suited with less wasteful tokens that are likely to be discarded on rollback.

2.3 Multi-Token Joint Speculative Decoding (MJSD)

Multi-Token Joint Speculative Decoding (MJSD) extends speculative decoding by improving both output quality and efficiency through joint token verification. Traditional speculative decoding relies on a two-model architecture, where a smaller draft model generates tokens sequentially and a larger target model verifies them in parallel. While this approach reduces latency, it maintains the same sampling distribution as greedy decoding and does not improve output quality.

MJSD addresses this limitation by introducing joint likelihood-based decoding. Instead of evaluating tokens independently, MJSD generates multiple tokens simultaneously and evaluates them based on their joint probability. This is motivated by Multi-Token Joint Greedy Decoding (MJGD), which demonstrates that considering multiple tokens jointly leads to improved perplexity and better sequence-level quality.

To make this approach practical, MJSD approximates the joint distribution of the target model using a smaller draft model and applies beam decoding to efficiently generate candidate token sequences. During verification, the target model evaluates these sequences in parallel and accepts the longest prefix of tokens that satisfies a joint likelihood criterion. This allows MJSD to accept longer token sequences per iteration compared to standard speculative decoding, increasing throughput.

Experimental results show that MJSD improves output perplexity by over 21% compared to greedy decoding while achieving a 2.21× speedup and significantly reducing energy consumption. :contentReference[oaicite:0]index=0

2.4 AMUSD

AMUSD proposes a system that speeds up token generation through combining the draft and verification phases into one asynchronous approach. This allows for parallel execution of both verification and drafting on separate devices as processes. A shared memory is utilized for asynchronous communication between both processes where a tensor for both phases is stored. The length of the draft process's tensor expands with each new draft token generated while the verification tensor increases with each new draft token being accepted. This resulted in a 29

2.5 Token Map Drafting

Token Map Drafting changes the traditional speculative decoding architecture by replacing the draft model with a token map. Rather than generating tokens with a draft model, it selects tokens from the precomputed token map. The token map acts as a dictionary with key-value pairs, the keys being an n-gram sequence and the values being a list of token sequences. It analyzes domain-specific text corpus and generates n-gram token sequences along with a probability values for each token in the n-gram sequence. It reduces the amount of

computational overhead without compromising decoding accuracy. By performing the token map drafting on the CI-AVSR dataset, it demonstrated speed-ups of around 1.27x along with a 1.37x speedup on their own internal dataset, showcasing its ability to outperform traditional speculative decoding models without sacrificing accuracy.

3 System Model

4 Methodology

Our system implements and evaluates speculative decoding through a modular framework consisting of a baseline decoder, a speculative decoder with verification and rollback, a KV-cache strategy, and a multi-token pipeline. Each component is designed to model different aspects of inference efficiency and system behavior.

4.1 Baseline Decoder

We first implement a baseline autoregressive decoder that generates tokens sequentially using a single model. Each token is produced through a full forward pass, establishing a reference for correctness and performance. This baseline is used to compare throughput and validate that speculative methods maintain consistent output behavior.

4.2 Speculative Decoder

The speculative decoder introduces a two-model architecture consisting of a draft model and a target model. The draft model generates a sequence of k speculative tokens, which are then verified by the target model in a single forward pass.

During verification, tokens are compared sequentially. Matching tokens are accepted, while the first mismatch triggers rollback. The accepted prefix is committed to the output, and the correct token from the target model is inserted. This process reduces the number of expensive target model evaluations by amortizing verification over multiple tokens.

4.3 KV-Cache Strategy

To support efficient verification and rollback, we implement a KV-cache simulation that separates tokens into committed and speculative states. Draft tokens are stored in the speculative state until verification occurs.

During verification, the longest matching prefix is identified and committed, while all remaining speculative tokens are discarded. This is implemented through explicit `commit` and `rollback` operations. This separation ensures correctness while avoiding unnecessary recomputation of verified tokens.

4.4 Multi-Token Pipeline

To improve utilization, we implement a multi-token pipeline that overlaps draft generation and verification. The system maintains a queue of speculative batches, where each batch contains a sequence of draft tokens.

The draft stage continuously generates batches and pushes them into the queue, while the verification stage processes the oldest batch using the target model. During verification, tokens are accepted until a mismatch occurs. The accepted prefix is committed, and any remaining speculative tokens are discarded. Additionally, all younger speculative batches in the pipeline are flushed to maintain correctness.

This design enables parallelism between drafting and verification while modeling realistic pipeline hazards such as rollback propagation and pipeline bubbles.

4.5 Adaptive Speculation Depth

We incorporate an adaptive speculation depth strategy that adjusts the number of draft tokens k based on observed acceptance behavior. When acceptance rates are high, the system increases k to maximize throughput by committing longer token sequences. When acceptance rates decrease, k is reduced to limit rollback overhead.

This adaptive mechanism allows the system to dynamically balance between aggressive speculation and conservative execution, improving efficiency across varying workloads.

4.6 Hardware-Assisted Rollback

Although our implementation performs rollback in software, we model a hardware-inspired rollback mechanism to improve efficiency. Speculative tokens are conceptually stored in a buffer, with commit and rollback pointers tracking verified and invalid regions.

Upon mismatch, speculative tokens beyond the rollback point are discarded in a single operation rather than recomputed. This mirrors our pipeline flushing behavior, but highlights how hardware support could further reduce overhead and pipeline stalls in future systems.

4.7 Baseline Decoder

4.8 Speculative Decoder

Compared to traditional autoregressive decoders, speculative decoding can generate multiple tokens at a time using two models: a draft model and a target model. The draft model performs what is called a draft generation where given an input sentence, it tokenizes each word in the sentence and is fed into the draft model. The draft model then proposes K draft tokens, each draft token being a word that completes the prompt along with a probability from the draft

model's probability distribution. The draft tokens and input sentence is then fed into the target model where it performs parallel verification. The target model takes in the draft tokens and input sentence and outputs its own probability distributions for the draft tokens in one single forward pass. Afterwards, each draft token's probability from the target and draft models is compared with each other and the following acceptance logic is proposed: If the probability of the target model is bigger than the draft model's probability, the token is accepted, meaning that it fits the probability distribution of the target model and it iterates to the next draft token. If the token is rejected, the rollback mechanism is activated, where all subsequent draft tokens are rejected and the corrected token is sampled from the corrected probability distribution and added to the input sentence. The input sentence is then fed back into the draft model where it proposes another K tokens and the process is repeated again until the full output is generated.

4.9 KV-Cache Strategy

We implement a KV-cache simulation that separates tokens into two states: committed tokens and speculative tokens.

During draft generation, tokens are stored in the speculative state. During verification, the target model compares speculative tokens against its own predictions and determines the accepted prefix. Accepted tokens are committed to the final sequence, while all remaining speculative tokens are discarded.

This behavior is implemented through explicit `commit` and `rollback` operations. The commit operation appends verified tokens to the committed state, while rollback clears all speculative tokens after each verification step. This ensures correctness while avoiding recomputation.

4.10 Multi-Token Pipeline

We implement a multi-token pipeline that overlaps draft generation and verification to improve hardware utilization.

The system maintains a queue of speculative batches. The draft stage generates batches of k tokens and pushes them into the queue, while the verification stage processes the oldest batch in the queue using the target model.

During verification, tokens are accepted sequentially until the first mismatch. The accepted prefix is committed to the output, and if a mismatch occurs, the correct token is inserted. All younger speculative batches in the queue are flushed to maintain correctness.

This design enables parallelism between draft and verification stages while ensuring that only verified tokens are committed, reducing idle time and improving throughput.

4.11 Adaptive Speculation Depth

While larger speculation depth k risks larger impact of rollbacks, too small of a speculation depth k increases target model overhead. To find the right k with greatest improvement to token throughput, either a mathematical analysis needs to be performed, or through expensive trial and error. Adaptive Speculation Depth automates the fine tuning of k where greater confidence in draft tokens increase k and vice versa. Acceptance rate is the metric for confidence, where too low of an acceptance rate decrease k to decrease the likely impact of rollbacks, leading to less wasted draft tokens. However, if acceptance rate is high, k is increases without fear of rollback wastage, allowing the single pass of the target model to verify more tokens, ultimately decreasing overhead and improving speed.

4.12 Hardware-Assisted Rollback

While our implementation performs rollback in software, we draw inspiration from hardware-assisted rollback mechanisms to improve efficiency. In speculative decoding, incorrectly predicted tokens must be discarded when verification fails, which can introduce overhead if handled through full recomputation.

Conceptually, this process can be modeled using a speculative buffer that stores draft tokens prior to verification, along with a commit pointer and rollback pointer. The commit pointer tracks the last verified token that is safe to retain, while the rollback pointer identifies where incorrect speculative tokens begin. During verification, matching tokens are committed, and a mismatch triggers rollback.

Instead of recomputing the sequence, all speculative tokens beyond the rollback point can be discarded in a single operation. This mirrors how our software pipeline flushes speculative batches upon mismatch, but a hardware implementation could perform this more efficiently at the architectural level.

By treating rollback as a control operation rather than recomputation, hardware-assisted designs have the potential to reduce overhead, minimize pipeline stalls, and further improve throughput in speculative decoding systems.

5 Performance Model

The performance model takes the throughput metrics from the individual models using baseline decoders to predict the speedup of the usage of those models in speculative decoding. By calculating the ratio of throughput between the draft model and token model we yield a coefficient β. Then by caclulating the expectation of the number of output tokens using speculative decoding and multiplying that with the cost of the round in terms of target model tokens using β, we find the speedup multiplier. The speedup multiplier is multiplied with the throughput of the target model to find the expected throughput of the speculative decoding model.

To evaluate the efficiency of our speculative decoding framework, we conducted a series of benchmarks comparing baseline autoregressive models against our speculative implementation across varying sequence lengths.

5.1 Model Selection

We utilize a two-model hierarchy for our speculative decoder to analyze the trade-off between speed and output correctness:

- **Draft Model:** TinyLlama-1.1B, chosen for its low parameter count and high inference speed during token proposal.
- **Target Model:** Llama-2-7b, serving as the larger, target model used to verify speculative tokens. This was chosen within our device constraints as a lack of resources for larger models.
- **Baselines:** Standard autoregressive decoding was performed on both TinyLlama and Llama-2-7b to establish performance floors and ceilings.

5.2 System Configuration

Our implementation incorporates several optimization strategies designed to reduce latency and improve hardware utilization:

- **KV-Cache Management:** We implement separate committed and speculative states. Draft tokens are stored in a temporary speculative buffer, which is either moved to the committed state upon successful verification or discarded during a rollback.
- **Adaptive Speculation Depth:** Rather than using a fixed speculation depth (k), the system dynamically adjusts based on the acceptance rate. We initialize k at 10% of the sequence length; the depth slowly increases when the acceptance rate is high and aggressively decreases when rejections occur.
- **Acceptance Logic:** We utilize random sampling for the verification stage to maintain the target model's original output distribution while maximizing the acceptance rate.

5.3 Metrics and Evaluation

Performance was measured across sequence lengths (n) of 50, 100, and 200 tokens to evaluate scaling over longer sequence lengths. The framework is evaluated using the following primary metrics:

- **Token Throughput:** Measured in tokens per second (t/s).
- **Speedup Ratio:** The ratio of speculative decoding throughput relative to the baseline Llama-2-7b throughput.

- **Acceptance Rate (α):** The ratio of draft tokens accepted by the target model to the total number of draft tokens generated.

To verify correctness, all tests were prompted with common sequences (e.g., "The first digits of pi are") to ensure the speculative output remained bit-for-bit identical to the baseline Llama-2-7b output.

6 Results

We evaluate the multi-token pipeline using TinyLlama-1.1B as the draft model and LLaMA2-7B as the target model. The pipeline overlaps draft and verification stages to improve utilization.

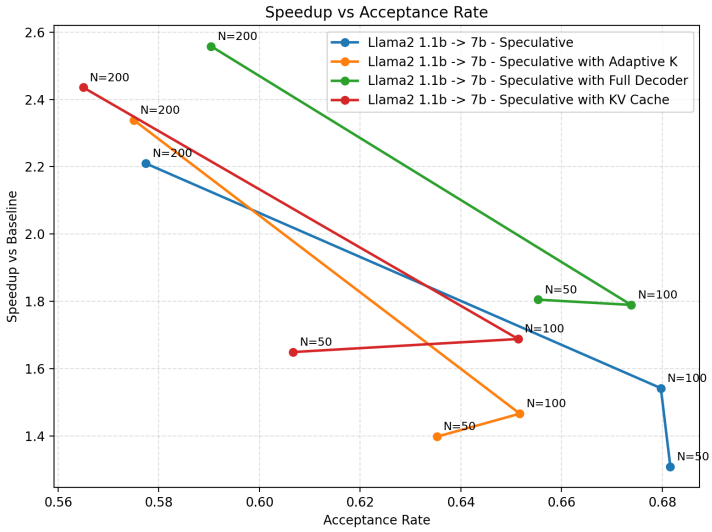

Figure 1: Speedup vs Acceptance Rate across speculative decoding configurations

The results show a strong correlation between acceptance rate and performance. Configurations with higher acceptance rates achieve greater speedup, as fewer rollbacks occur and more speculative tokens are successfully committed.

The full decoder configuration achieves the highest speedup (up to approximately 2.5x), while basic speculative decoding performs worse due to lower acceptance efficiency. KV-cache and adaptive strategies improve performance by reducing redundant computation and adjusting speculation depth.

These results highlight that pipeline effectiveness depends heavily on acceptance behavior. When acceptance is high, the pipeline benefits from overlapping computation. However, when acceptance drops, rollback overhead and pipeline flushing reduce overall gains.

7 Analysis

8 Conclusion

In this work, we implemented and analyzed speculative decoding as a practical approach to accelerating large language model inference. By combining a draft model for token generation with a target model for verification, we demonstrated how speculative execution can reduce the number of expensive forward passes while maintaining high output fidelity.

We extended the baseline speculative decoding framework with several system-level improvements, including a KV-cache strategy for managing committed and speculative states, a multi-token pipeline that overlaps draft and verification stages, and an adaptive speculation depth mechanism guided by observed acceptance rates. Together, these components improve hardware utilization and reduce idle time during inference.

Our results show that performance gains are highly dependent on acceptance behavior. Higher acceptance rates lead to longer committed token sequences per iteration, enabling greater throughput and more effective pipeline utilization. However, when acceptance rates decrease, rollback overhead and pipeline flushing introduce inefficiencies that limit speedup.

Overall, this project highlights the importance of system design in speculative decoding. While the core algorithm provides a foundation for acceleration, practical performance depends on how efficiently drafting, verification, and rollback are coordinated. Future work could explore improved acceptance strategies, better draft model alignment, and hardware-aware optimizations to further enhance throughput and scalability.

References

[1] Z. Zhao, Z. Lin, Y. Zhao, Q. Li, G. Huang, and J. Yan, "DySpec: Faster speculative decoding with dynamic token tree structure," *arXiv preprint arXiv:2402.05370*, 2024.

[2] X. Miao *et al.*, "SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification," in *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '24)*, 2024, pp. 314–329.

[3] T. V. Ho, H. Kokubo, M. Yamamoto, and Y. Kawaguchi, "Model-Free Speculative Decoding for Transformer-Based ASR with Token Map Drafting," in *2025 33rd European Signal Processing Conference (EUSIPCO)*, Palermo, Italy, 2025, pp. 361–365, doi: 10.23919/EUSIPCO63237.2025.11226729.

[4] B. McDanel, "AMUSD: Asynchronous Multi-Device Speculative Decoding for LLM Acceleration," in *2025 IEEE International Symposium on Cir-*

cuits and Systems (ISCAS), London, United Kingdom, 2025, pp. 1–5, doi: 10.1109/ISCAS56072.2025.11043575.

[5] Z. Qin, Y. Ge, J. Yan, Y. Wang, and G. Huang, "Multi-Token Joint Speculative Decoding for Accelerating Large Language Model Inference," *arXiv preprint arXiv:2407.09722*, 2024.

[6] M. Yan, S. Agarwal, and S. Venkataraman, "Decoding Speculative Decoding," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2025, pp. 6460–6473.

[7] J. Liu *et al.*, "Decoding Speculative Decoding," *arXiv preprint*, 2024.